

Multi – level linked list
–Delete At End
–Space Complexity and Algorithm

Program: –

```
void deleteAtEnd()
{
    if (head == nullptr)
        return;

    if (head->next == nullptr)
    {
        free(head);
        head = nullptr;
        return;
    }

    Node *temp = head;
    while (temp->next->next != nullptr)
    {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = nullptr;
    cout << "Deleted from end." << endl;
}

int main()
{
    int choice;
    while (true)
    {
        cout << "6. Delete at End (Main)" << endl;
        cin >> choice;
```

```
    switch (choice)
    {
    case 6:
        deleteAtEnd();
        break;
    default:
        cout << "Invalid choice" << endl;
    }
}
return 0;
}
```

Space Complexity Analysis

1. Input Space Complexity

A. Parameter – Only Input Space Complexity

Function signature:

void deleteAtEnd()

There are:

- ***no parameters***
- ***no arrays***
- ***no passed linked list pointer***

So under the parameter – only definition:

<i>$O(1)$</i>

Why?

Because parameter memory does not grow with n .

Even if the linked list has 1 node or 1 million nodes:

- *function parameters remain zero*

B. Full Data Definition / Holistic Input Space Complexity

Now the actual input is the entire multilevel linked list.

Each node contains:

```
int data;  
struct Node *next; // Points to the right (Same Level)  
struct Node *child; // Points down (Sub Level)
```

So each node occupies memory for:

- *one integer*
- *two pointers*

Total Input Memory

If there are n total nodes:

$$n \times (\text{size of int} + \text{size of pointer})$$

Therefore total input memory grows linearly with n .

Full/Holistic Input Space Complexity:

$$O(n)$$

Why?

Because the algorithm operates on an entire linked list of size n.

Even though nodes were created before the function call, they still constitute the algorithm's input.

2. Auxiliary Space Complexity

Auxiliary Space Complexity measures:

Extra temporary memory used by the algorithm excluding input storage.

```
Node *temp
```

Only one temporary pointer variable is created.

Memory Used

The function uses:

- ***one pointer variable (temp)***
- ***no arrays***
- ***no recursion stack***
- ***no dynamic allocation (malloc/new)***
- ***no additional linked list nodes***

Important Observation

This loop:

```
while (temp->next->next != nullptr)
{
    temp = temp->next;
}
```

traverses the list, but traversal does not increase memory usage.

It only reuses the same pointer variable repeatedly.

Auxiliary Space Complexity

$O(1)$

(Constant extra space)

Why free() Does NOT Increase Auxiliary Space

```
free(temp->next);
```

actually releases memory.

It does not allocate new memory.

Therefore it does not contribute positively to auxiliary space.

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\text{Auxiliary}} + \underbrace{O(1)}_{\text{Input}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\text{Auxiliary}} + \underbrace{O(n)}_{\text{Input}} = O(n).$$

1) If Asked “Total Space Complexity” then natural answer is
: $O(n)$ [*Full Data + Auxiliary*]

2) *If not asked total space complexity then:*

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

In most academic DSA contexts and coding interviews, when someone simply asks:

“What is the space complexity?”

they usually mean:

Auxiliary Space Complexity

unless they explicitly say:

- total space complexity
- overall memory usage
- data structure space
- input space
- storage complexity

So for your insertion function, the expected answer is usually:

$O(1)$

because only constant extra working memory is used.

Algorithm:

MultiDelAtEnd():

1. [*CHECK IF LIST IS EMPTY*]
If START = NULL, then:

Return void and EXIT.

[*End of If structure*]

2. [*If List Contain 1 node*]

if LINK[START] = NULL, then:

free(start) [Single Node connected with Head , gets freed]

SET START := NULL [List becomes Empty]

Return void and exit.

[*End of If structure*]

[*if List has numerous nodes*]

3. *SET TEMPPTR := START [TEMP pointer point to the First Node]*

[*Traverse Node till Temp pointer*
points to n – 1 node]

4. *Repeat while(Link[Link[TEMPPTR]] ≠ NULL):*
SET TEMP := LINK[TEMPPTR]
[END OF LOOP]

5. *Call FREE(LINK[TEMPPTR])[FREE memory allocated by nth node]*

6. *SET LINK[TEMPPTR] := NULL* [*AS NEXT of NODE pointed by TEMP*
pointer set to NULL , it becomes
last node.]

7. [*END OF ALGORITHM*]
-